

103 lines

```

1 """
2 07_capture_dashboard.py
3 =====
4 Captures real screenshots of the interactive dashboard for use in the
5 presentation deck and the report.
6
7 Uses Playwright + Chromium against a local static server so the shots are of
8 the genuine running page (not a mock-up). Skips gracefully if Playwright is
9 not installed - no other deliverable depends on it.
10
11 Outputs
12 -----
13 figures/dashboard/dash_scenarioC_light.png
14 figures/dashboard/dash_scenarioC_dark.png
15 figures/dashboard/dash_reasoning.png
16 """
17
18 from __future__ import annotations
19
20 import subprocess
21 import sys
22 import threading
23 import time
24 from pathlib import Path
25
26 ROOT = Path(__file__).resolve().parents[1]
27 OUT = ROOT / "figures" / "dashboard"
28 PORT = 8731 # dedicated port so it never collides with the dev preview server
29
30
31 def serve():
32     return subprocess.Popen(
33 [sys.executable, "-m", "http.server", str(PORT), "--directory", str(ROOT / "dashboard")],
34     stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
35 )
36
37
38 def main():
39     try:
40         from playwright.sync_api import sync_playwright
41     except ImportError:
42         print(" Playwright not installed - skipping dashboard capture.")
43         print(" (pip install playwright && playwright install chromium)")
44         return
45
46     OUT.mkdir(parents=True, exist_ok=True)
47     srv = serve()
48     time.sleep(1.5)
49     url = f"http://localhost:{PORT}/index.html"
50
51     try:
52         with sync_playwright() as pw:
53             browser = pw.chromium.launch()
54
55             def shot(name, theme, setup_js, full=False, clip=None):
56                 page = browser.new_page(viewport={"width": 1600, "height": 1000},
57                                         device_scale_factor=2,
58                                         color_scheme=theme)
59                 page.goto(url, wait_until="networkidle")
60                 # Headless Chromium composites `backdrop-filter` layers in a way
61                 # that blanks out <canvas> children in screenshots (the canvas
62                 # pixels are genuinely there - verified via getImageData - but
63                 # they do not make it into the captured frame). Disabling the
64                 # filter only for the capture gives a faithful shot of the
65                 # charts; the live page is unaffected.
66                 page.add_style_tag(content=(
67                     """{backdrop-filter:none!important;"""

```

```

68 "-webkit-backdrop-filter:none!important}"
69 # The rejected-candidate list scrolls on the live page; for a
70 # still image let it expand so no row is sliced in half.
71 ".reject-list{max-height:none!important;overflow:visible!important}"
72 ))
73 page.evaluate(
74 "t => { document.documentElement.dataset.theme = t; }", theme
75 )
76 page.evaluate(setup_js)
77 page.wait_for_timeout(1400)
78 path = OUT / name
79 if clip:
80 page.locator(clip).screenshot(path=str(path))
81 else:
82 page.screenshot(path=str(path), full_page=full)
83 print(f" wrote {path.relative_to(ROOT)}")
84 page.close()
85
86 run_c = """() => {
87 document.querySelector('[data-scenario="C"]').click();
88 const s = document.querySelector('#speed');
89 s.value = 'instant'; s.dispatchEvent(new Event('change'));
90 document.querySelector('#btn-play').click();
91 }"""
92
93 shot("dash_scenarioC_light.png", "light", run_c)
94 shot("dash_scenarioC_dark.png", "dark", run_c)
95 shot("dash_reasoning.png", "light", run_c, clip=".reasoning")
96 browser.close()
97 finally:
98 srv.terminate()
99
100
101 if __name__ == "__main__":
102     main()
103

```